# Error Detection

Error detection is the process of **identifying errors** that occur during the **transmission of data** over a communication channel. When data is sent from a sender to a receiver, noise or interference may alter some bits. Error detection ensures that the receiver can **detect whether the received data is correct or corrupted**.

**Single Bit Error**
A **single-bit error** occurs when **only one bit** in the transmitted data is altered (0 → 1 or 1 → 0) during transmission.

**Cause:**

- Minor noise in the communication channel
- Sudden interference

Sent: 10110010

Received: 10110000

↑

(Only 1 bit changed)

## Burst Error

**Definition:**
A **burst error** occurs when **two or more bits in the data unit are altered**, and these bits are **consecutive or grouped together**.

- The **length of the burst** = distance between the first and last error bit (including both).

**Cause:**

- Continuous interference
- Electrical spikes or channel noise
- Poor shielding or fading in wireless channels

Sent: 1011001110001110

Received: 1011111010001110

↑↑↑↑↑↑↑

(Burst error of length 7)

Error Detection Method

# Redundancy

**Definition:**
Redundancy is the technique of adding **extra bits** to the original data to help **detect or correct errors** during transmission. These extra bits do not carry useful information themselves; they are used solely for error checking.

**Explanation:**

- More redundancy improves the ability to detect and correct errors.
- The extra bits can be in the form of **parity bits, checksums, or CRC bits**.
- All error detection and correction methods are based on this concept.

**Example:**
If data = `1011` (4 bits) and we add 3 redundant bits for error checking → transmitted = `1011100`.

**Use:** Fundamental in all error detection and correction techniques.

---

# 2. Parity Check

**Definition:**
Parity check is a simple error detection method in which **one extra bit** (parity bit) is added to make the total number of 1s in the data **even (even parity)** or **odd (odd parity)**.

**Explanation:**

- Only detects **single-bit errors**.
- If two or more bits are flipped, parity check may fail.
- Can be applied **per byte** or per block of data.

**Example:**

- Data = `1010001` (3 ones → odd)
- Even parity → add 1 → `10100011`
- Odd parity → add 0 → `10100010`

**Use:** Used in early modems and simple communication systems.

---

# 3. Cyclic Redundancy Check (CRC)

**Definition:**
CRC is an error detection technique where the data is treated as a **binary polynomial**, divided by a **generator polynomial (G(x))**, and the **remainder** is appended as **CRC bits**.

**Explanation:**

- Very powerful and widely used.
- Detects **single-bit, double-bit, and burst errors**.
- Receiver divides the received data by the same generator; if remainder = 0 → no error.

**Example:**

- Data = `1101`, Generator = `1011`
- Append zeros (degree of generator − 1) → `1101000`
- Divide by 1011 → Remainder = `011`
- Transmit = `1101011`
- Receiver divides by 1011 → remainder 0 → ✅ no error

**Use:** Ethernet, USB, HDLC, and other high-reliability networks.

---

# 4. Checksum

**Definition:**
Checksum is an error detection method in which data is divided into **equal-size blocks**, all blocks are **added together using 1's complement arithmetic**, and the **1's complement of the sum** is sent as the checksum.

**Explanation:**

- Receiver adds all blocks including the checksum.
- If the result is all 1s → data is correct.
- Detects **burst errors** effectively.

**Example:**

- Data blocks (8 bits each): `10101001` + `00111001`
- Sum = `11100010`
- 1's complement → `00011101` (checksum)
- Transmit: `10101001 00111001 00011101`
- Receiver adds all → `11111111` → ✅ No error

**Use:** TCP, UDP, IP protocols.

# Single (1-Dimensional) Parity Check

### Definition:

A single parity check adds **one extra parity bit** to a group of data bits to make the number of 1s either **even (even parity)** or **odd (odd parity)**.

### How It Works:

1. Count the number of 1s in the data.
2. Add a parity bit:
   - Even parity → total number of 1s becomes even
   - Odd parity → total number of 1s becomes odd

### Example:

Data = `1010001` (3 ones → odd)

- **Even parity:** add 1 → `10100011`
- **Odd parity:** add 0 → `10100010`

**Detection:**

- Receiver checks parity:
  - If parity condition satisfied → no error
  - If violated → error detected

✅ Detects **single-bit errors**
❌ Cannot detect multiple-bit errors if an even number of bits change.

---

# 2. Two-Dimensional (Block) Parity Check

### Definition:

Two-dimensional parity check is an **extension of single parity** where parity bits are added for both **rows and columns** in a **block of data**.

- The data is arranged in a **matrix (rows × columns)**.
- Each **row** has a **row parity bit**.
- Each **column** has a **column parity bit**.

### How It Works:

1. Arrange data in rows.
2. Add **row parity bits** (like single parity).
3. Add **column parity bits** to the columns.

4. Transmit the matrix including row and column parity bits.

| Bit1 | Bit2 | Bit3 | Bit4 |
|---|---|---|---|
| 1 | 0 | 1 | 1 |
| 0 | 1 | 0 | 0 |
| 1 | 1 | 0 | 1 |
| 0 | 0 | 1 | 1 |

- **Row parity:** add 1 bit to make 1s even/odd in each row.
- **Column parity:** add 1 bit for each column to make 1s even/odd.

### Detection and Correction:

- Can detect **single-bit and some multiple-bit errors**.
- Can locate **single-bit errors** (intersection of row & column parity failure).

**Example Scenario:**

- If a single bit flips in transmission:
  - Row parity of that row fails
  - Column parity of that column fails
    → Intersection identifies the erroneous bit → can **correct** it.

Q) Calculate checksum at sender and receiver block of 16 bits is to be sent using a checksum of 8 bits (10101001 00111001)

Block of 16 bits = `10101001 00111001`
Checksum = 8 bits

So, we divide the data into **two 8-bit words**:

```
Word 1: 10101001
Word 2: 00111001
```

---

# ✅ At Sender Side

### Step 1: Add both 8-bit words (using binary addition).

```
  10101001
+ 00111001
------------
  11100010
```

No carry beyond 8 bits → so the sum = `11100010`

### Step 2: Take 1's complement of the sum to get the checksum.

```
Sum:        11100010
1's comp.:  00011101
```

✅ **Checksum = 00011101**

---

Step 3: Sender transmits:

```
Data: 10101001 00111001
Checksum: 00011101
```

---

# ✅ At Receiver Side

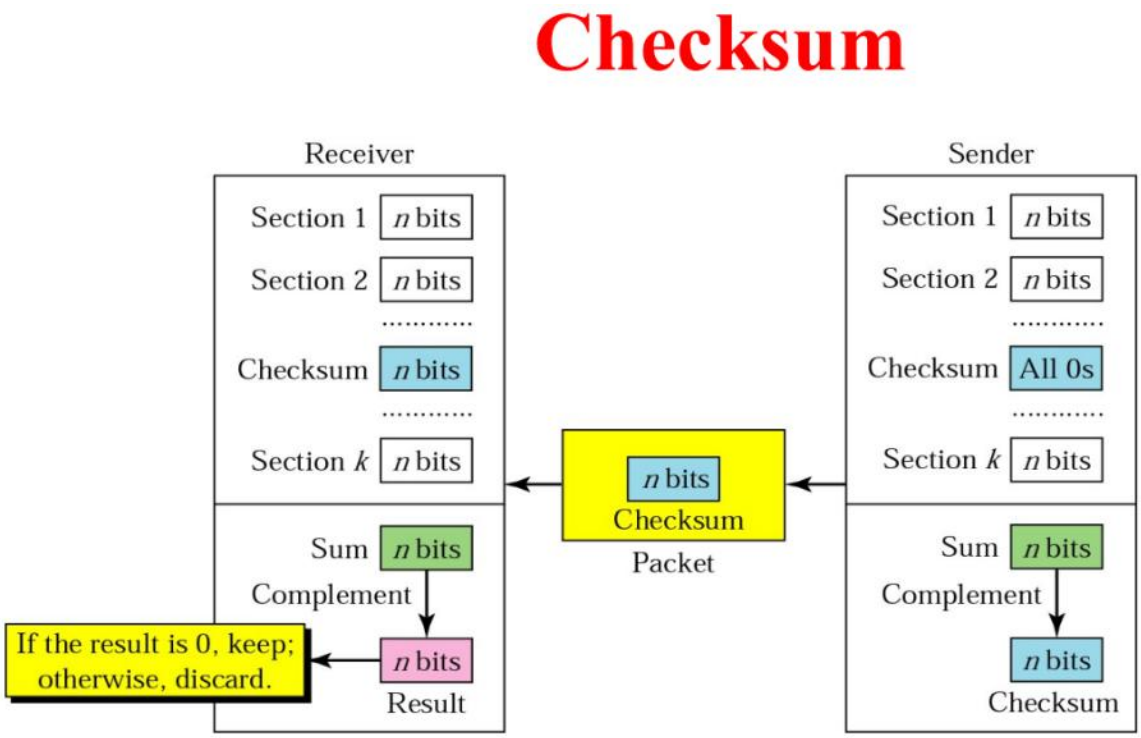
Receiver adds **all three 8-bit blocks** (two data blocks + checksum):

```
  10101001
+ 00111001
------------
  11100010

+ 00011101
------------
  11111111
```

Step 4: Result = `11111111` (all 1's)

✅ Therefore, **no error detected**.



## *The sender follows these steps:*

- *The unit is divided into k sections, each of n bits.*
- *All sections are added using one's complement to get the sum.*
- *The sum is complemented and becomes the checksum.*
- *The checksum is sent with the data.*

## *The receiver follows these steps:*

- *The unit is divided into k sections, each of n bits.*
- *All sections are added using one's complement to get the sum.*
- *The sum is complemented.*
- *If the result is zero, the data are accepted: otherwise, rejected.*